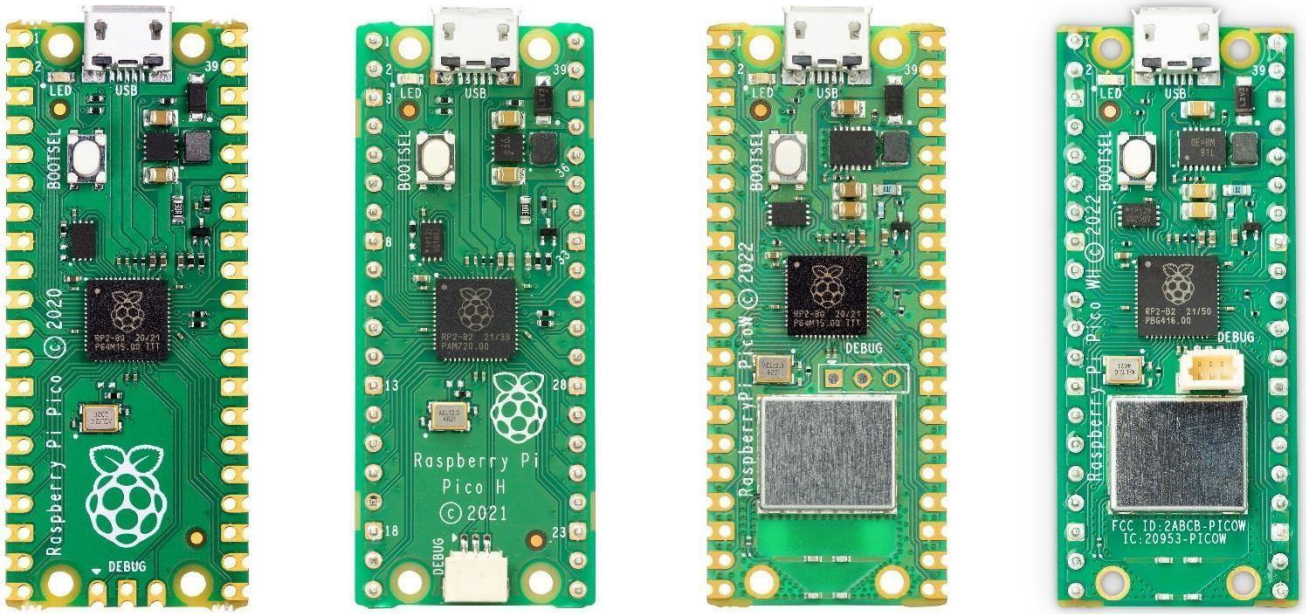
Unit IV

IoT implementation using Pico W board

TY MDM-III: IOT implementation using Raspberry Pi Boards



Sensors

A sensor is a device that detects and measures a physical property or environmental condition such as temperature, humidity, soil moisture, or light and then converts this information into a signal that can be read, recorded, or processed by a system like a microcontroller or an IoT device.

Examples of Sensors:

- Temperature Sensors: Measure air, soil, or plant temperature critical for optimal crop growth.
- Humidity Sensors: Monitor air and soil moisture levels to maintain ideal hydration for plants.
- Soil Moisture Sensors: Measure the water content in soil, helping optimize irrigation to avoid over- or under-watering.

- pH Sensors: Analyze soil acidity or alkalinity, which affects nutrient availability and crop health.
- Light Sensors: Monitor ambient light levels for crops, important in greenhouses and farming under controlled lighting.
- Location Sensors (GPS): Track machinery, field boundaries, and spatial variations to optimize planting and resource usage.
- Gas Sensors: Detect gases like CO2 and ethylene related to plant respiration and disease.
- Pest and Disease Monitoring Sensors: Use image recognition or acoustic detection to identify threats early.
- Crop Growth Sensors: Measure parameters like chlorophyll content, plant height, or fruit development for growth tracking.

Actuators

An actuator is a device that converts electrical signals into physical action or mechanical movement.

In the context of IoT and automation, actuators receive commands from a controller (often driven by sensor data) and perform real-world tasks like opening a valve, turning on a pump, or moving a robotic arm.

Examples of Actuators:

- Electric Actuators: Convert electrical energy into mechanical motion, frequently used for precise control such as solenoids, motors, and servo motors.
- Hydraulic Actuators: Use fluid pressure to generate large force and motion, common in heavy machinery but less typical in small-scale IoT agriculture.
- Pneumatic Actuators: Use compressed air or vacuum to create linear or rotary motion, valuable where low-maintenance and fast motion are needed.

- Linear Actuators: Produce motion in a straight line; commonly used for moving valves, gates, or openings.
- Relays: Electromechanical switches controlled by low-voltage signals to switch higher voltages on/off, widely used for appliance and irrigation control in agriculture.
- Solenoids: Electromagnetic actuators used for locking mechanisms, valves, and small mechanical actions.
- Thermal Actuators: Use heat-sensitive materials to create motion, such as in temperature-responsive valves.
- Magnetic Actuators: Convert electromagnetic energy into motion, used in precision applications.

Microcontroller (e.g., Raspberry Pi Pico W)

1. Reads sensor data
2. Controls actuators
3. Sends/receives data over Wi-Fi to cloud platforms

Example of Raspberry pi pico w with DHT11

- The DHT11 is a popular, low-cost digital temperature and humidity sensor that can be interfaced with the Raspberry Pi Pico W to monitor environmental conditions like temperature and humidity in real-time.
- Connecting DHT11 to Raspberry Pi Pico W
  - Power (VCC): Connect to the Pico's 3.3V pin (Pin 36).
  - Ground (GND): Connect to Pico's GND pin (Pin 38).
  - Data Pin: Connect to any GPIO pin on the Pico W (common choices are GPIO 16 or GPIO 22).

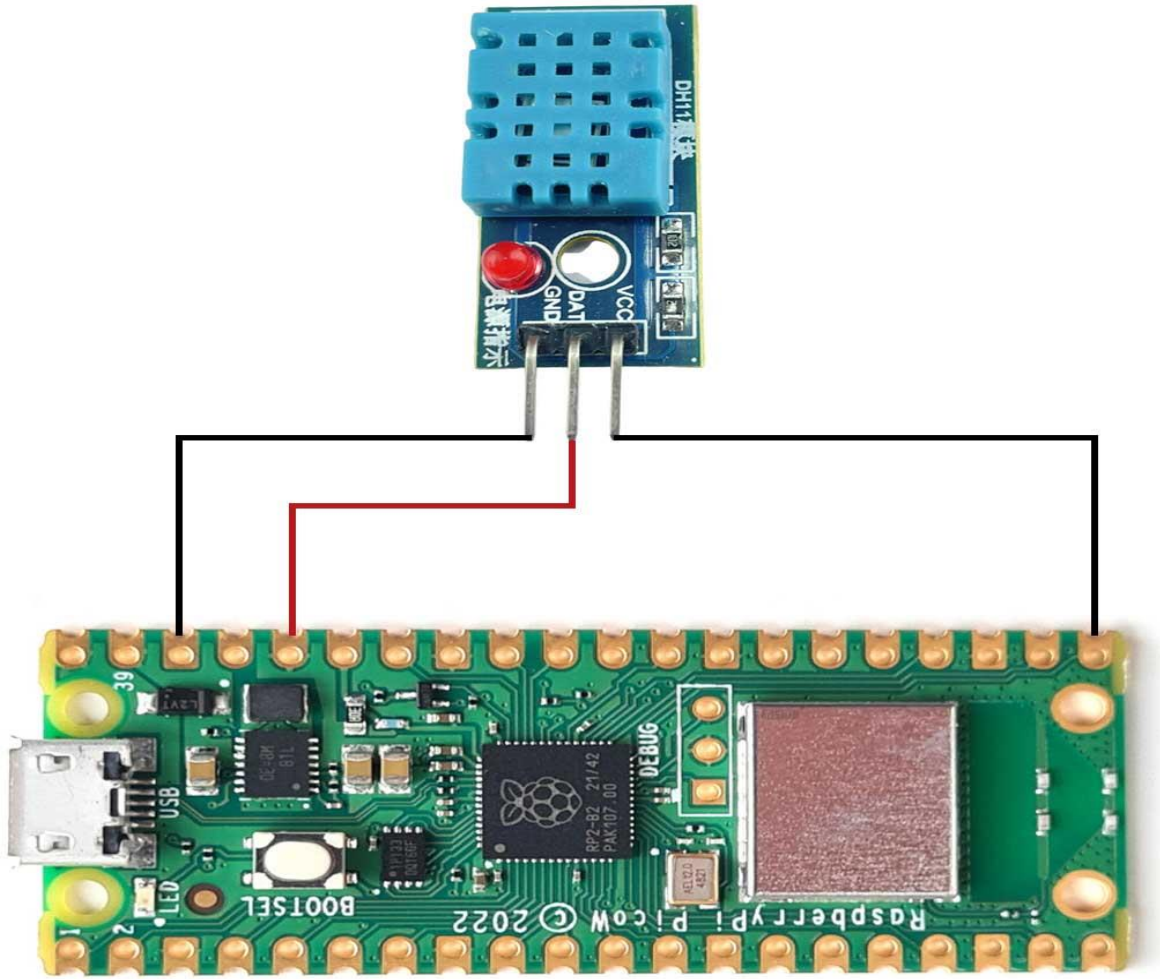
Code

```
import dht
from machine import Pin
import time

dht_sensor = dht.DHT11(Pin(16))  # Initialize DHT11 sensor on GPIO 16

while True:
    dht_sensor.measure()  # Measure temperature and humidity
    temp = dht_sensor.temperature()  # Get temperature in Celsius
    hum = dht_sensor.humidity()  # Get humidity percentage
    print("Temperature:", temp, "°C")
    print("Humidity:", hum, "%")
    time.sleep(1)  # Wait for 1 second before next reading
```

Interfacing Diagram of DHT 11 with Raspberry Pi Pico W



Cloud Platform (Ubidots, Blynk, ThingSpeak)

1. Receives sensor data
2. Visualizes data in dashboards
3. Allows remote control (e.g., switching devices on/off)
4. Sends alerts or automates actions

Implantation IoT systems for Home automation, Agriculture, Industry etc using Ubidots, Blynk, Thingspeak

- Implementing IoT systems across different sectors like home automation, agriculture, and industry involves using cloud platforms such as Ubidots, Blynk, or ThingSpeak to collect, analyze, and act on data from connected devices. Each platform offers unique features for building a robust and scalable solution.

Implantation IoT systems for Home automation using cloud platforms such as Ubidots, Blynk, Thingspeak

Home automation IoT systems using Raspberry Pi Pico W can be effectively implemented on cloud platforms like Ubidots, Blynk, and ThingSpeak to enable wireless monitoring and remote appliance control.

Each cloud service offers device dashboards, data collection, and mobile/web-based management, making real-time home automation practical and scalable.

Ubidots Integration

- Create a free Ubidots account and set up a device (e.g., "Pico W Home").
- Add "variables" on Ubidots representing each appliance or sensor, such as relay status or temperature readings.
- Use widgets (control switches, charts) in the dashboard to visualize data and send commands.
- Program Pico W using MicroPython (or Arduino API) to read sensor data (e.g., temperature, switches) and publish to Ubidots via REST API using the device token.
- Use received variable states to toggle GPIOs and control relays for lights, fans, etc., from anywhere in the world via dashboard switches.

Blynk Integration

- Sign up and create a Blynk project, add devices, and use the app to design a dashboard (with switches, buttons, sliders, etc.).
- Obtain Blynk authentication token, connect Pico W to Wi-Fi, and program with Blynk libraries for MicroPython.
- Use Blynk cloud to send control commands (turn appliances ON/OFF, adjust settings) and receive sensor data.
- Control 4-channel relays, lights, and servos using Blynk's virtual pins mapped to Pico W GPIOs; app widgets let users toggle appliances from mobile devices.

ThingSpeak Integration

- Set up a free account, channel, and API key on ThingSpeak for collecting and visualizing sensor data (temperature, humidity, status).
- Program Pico W in MicroPython to send HTTP POST requests with sensor readings and appliance status at intervals, using Wi-Fi connection for internet access.
- ThingSpeak dashboard visualizes incoming data over time, enabling monitoring and historical analysis (e.g., temperature fluctuations, on-off cycles).
- Pico W reads ThingSpeak channel status (optional) for actuator control, enabling basic remote execution. More advanced control uses MQTT for bi-directional communication.

Example Home Automation using Raspberry Pi Pico W & Blynk 2.0

Components Required:

1. Raspberry Pi Pico W Board
2. 4 channel Relay Module
3. Breadboard
4. Jumper Wires

Circuit Diagram

Connect the VCC pin of the relay module to the 5V pir of the Raspberry Pi Pico W.

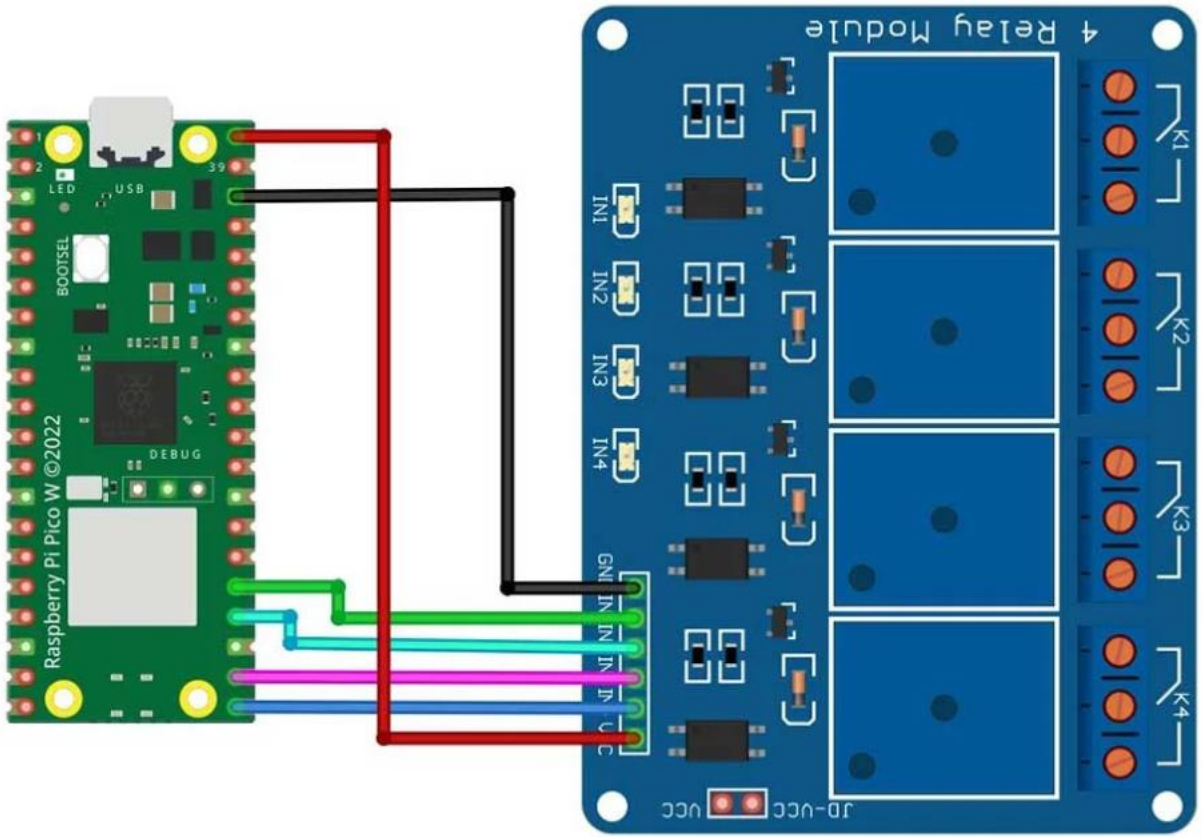
Connect the GND pin of the relay module to the GND pin of the Raspberry Pi Pico W.

Connect the IN1 pin of the relay module to the GP19 pin of the Raspberry Pi Pico W.

Connect the IN2 pin of the relay module to the GP18 pin of the Raspberry Pi Pico W.

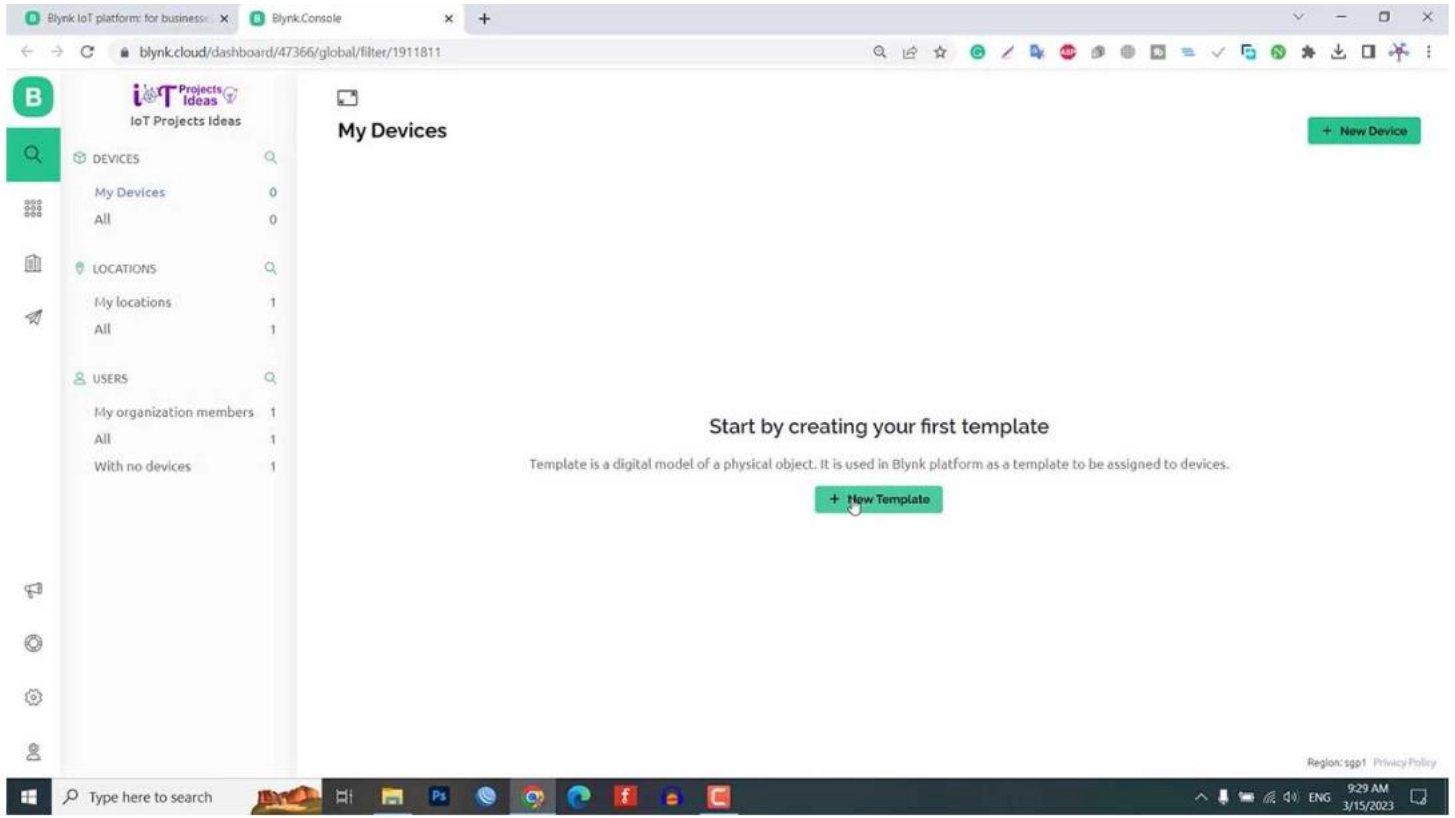
Connect the IN3 pin of the relay module to the GP17 pin of the Raspberry Pi Pico W.

Connect the IN4 pin of the relay module to the GP16 pin of the Raspberry Pi Pico W.

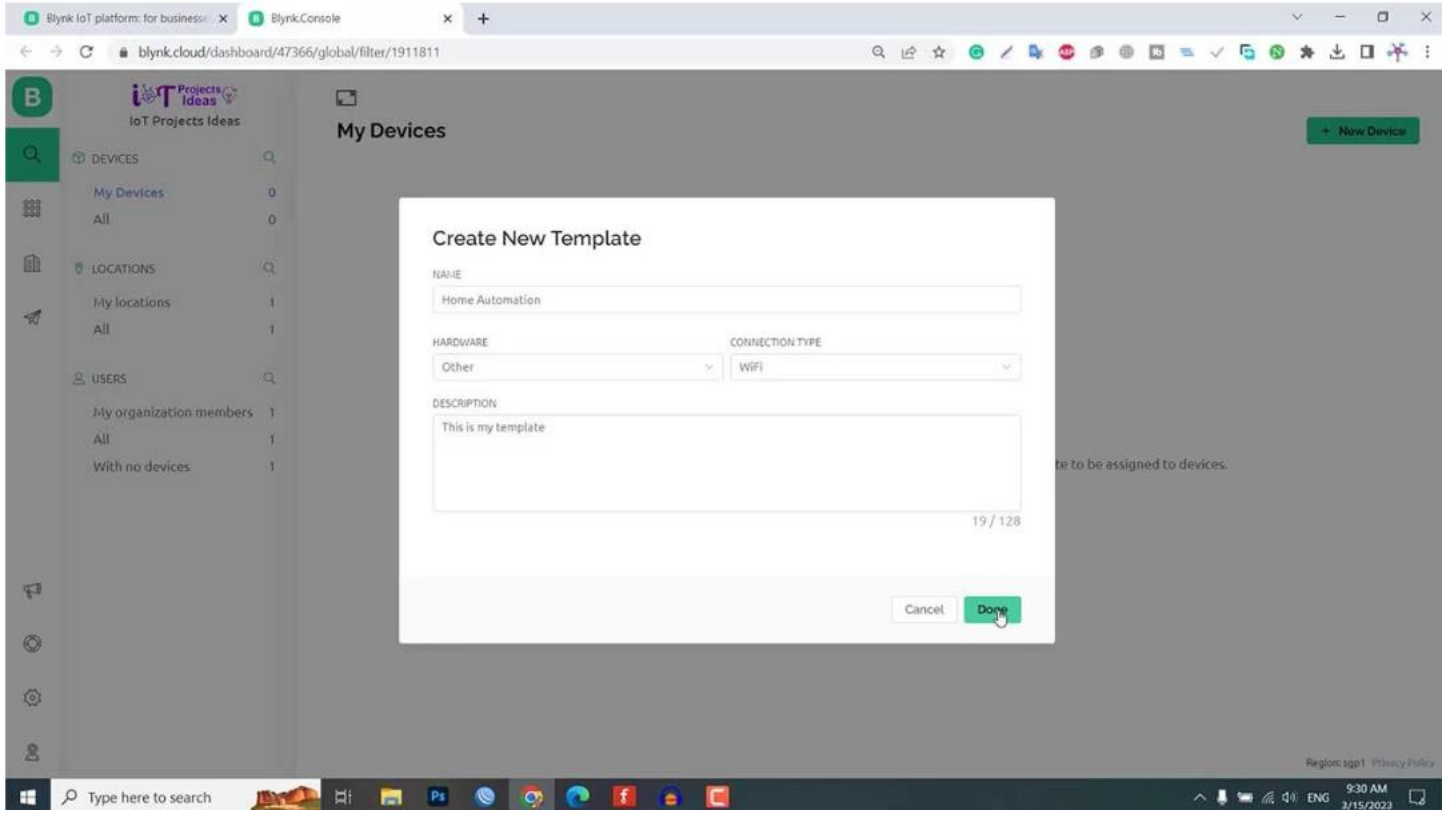


Steps for Blynk App

1. Go to https://blynk.cloud and either create a new Blynk account on the website or **sign in** using your registered email ID.
2. Click on "**Add New Template**".



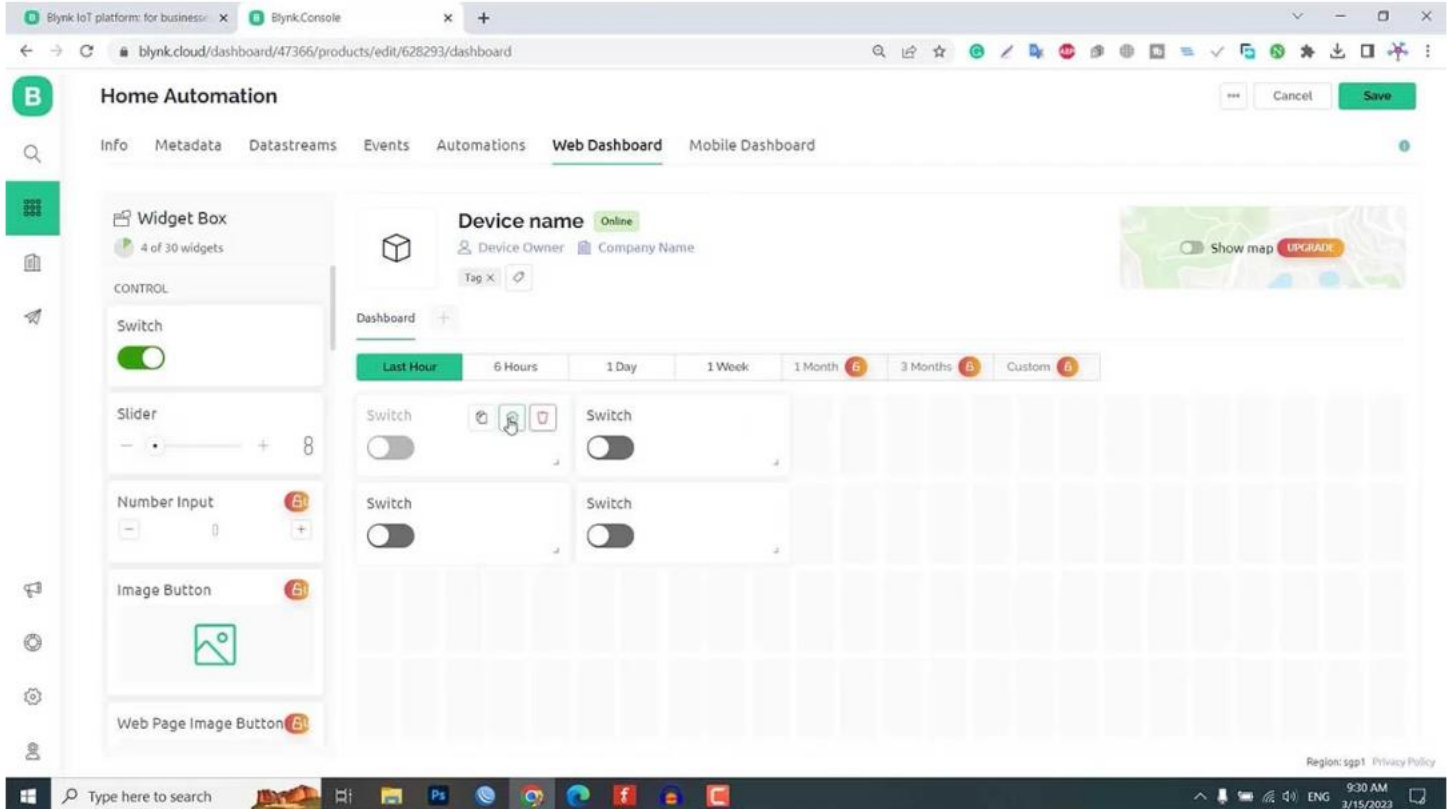
3. Provide a name for the project and select “**Other**” as the hardware type, with the connection type set to WiFi.



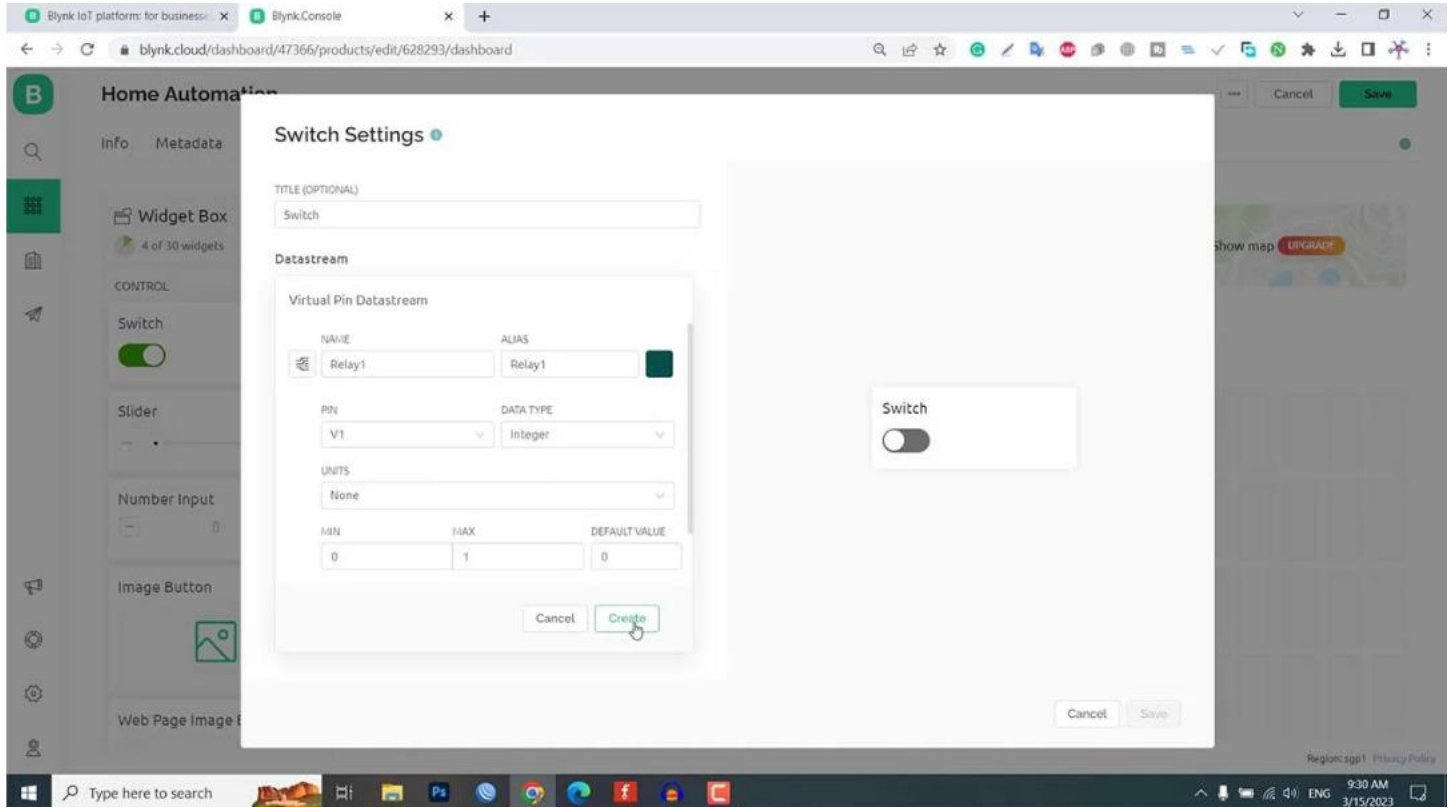
4. Click on **Done** so that the template is created successfully.

5. Configure Widgets with Datastream

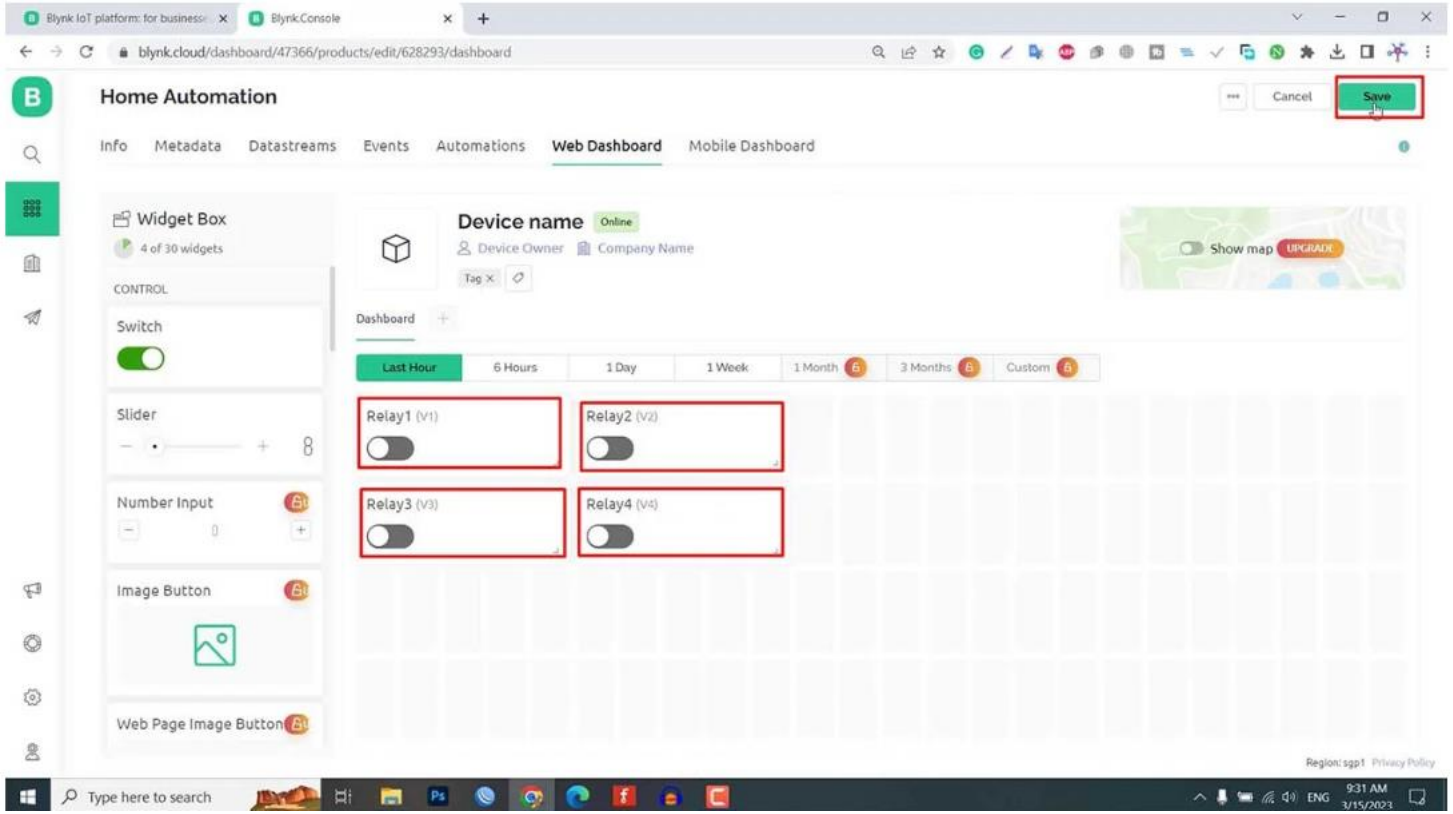
Using the web dashboard, you can create four switch widgets and configure them for relays.



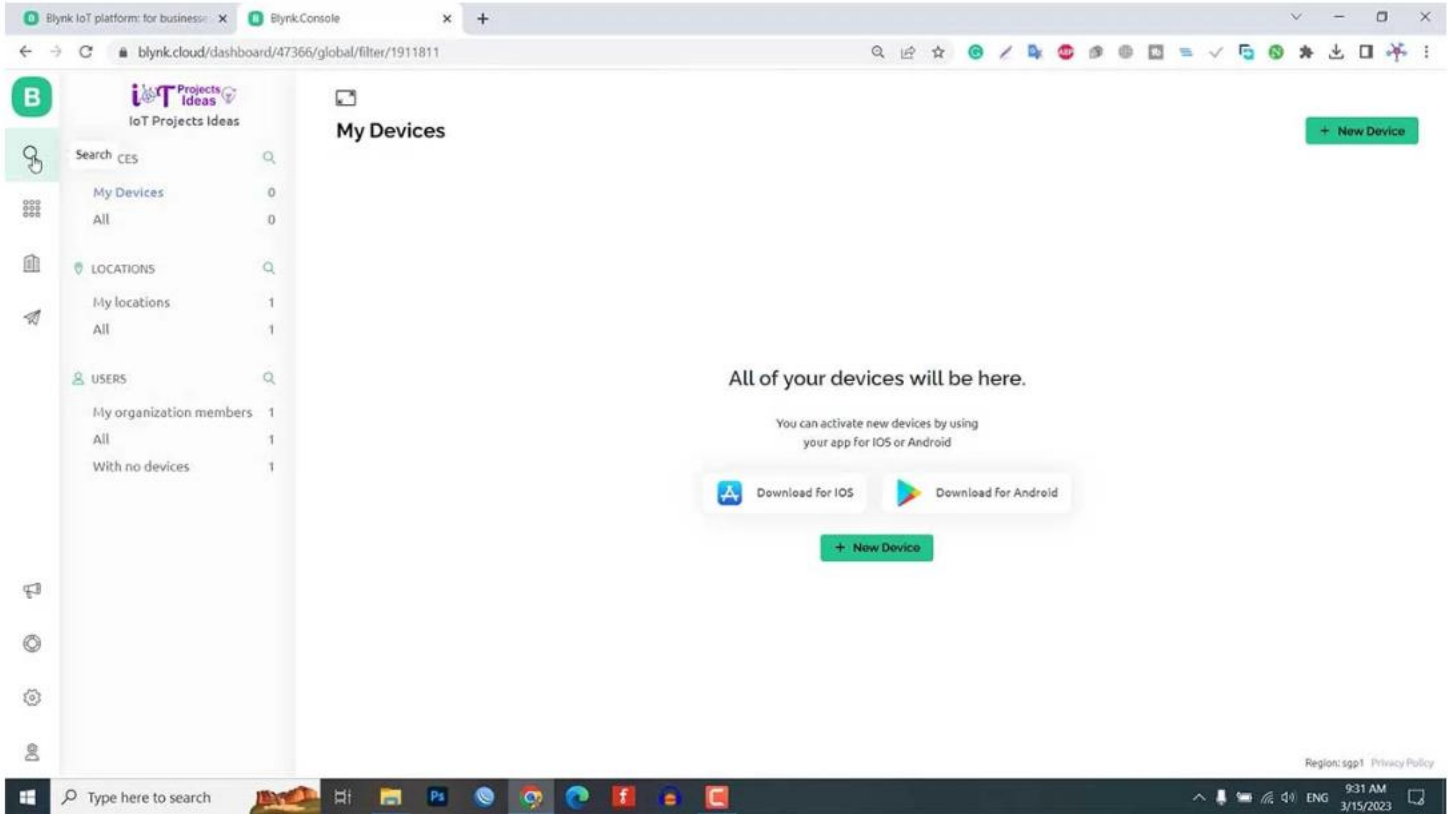
6. Click on the gear icon next to each button and **create a new datastream**. Choose the virtual pin and name the data stream “**Relay1**“, assigning it the **virtual pin V1** and setting the data type to integer.



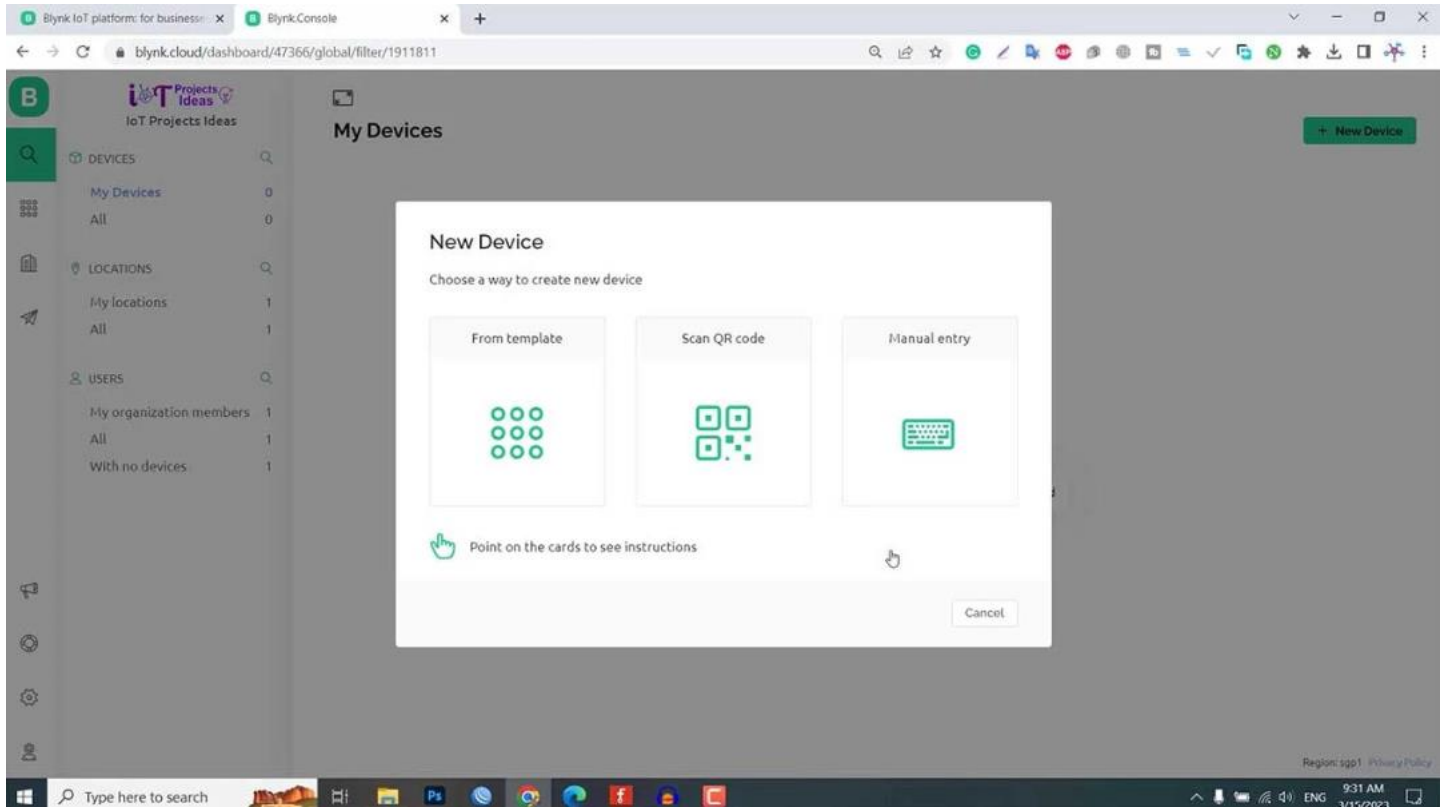
7. Repeat this process for the other three relays, assigning them to virtual pins **V2, V3, and V4**. Once all four datastreams have been created, click “**Save** ” to save the entire project settings.



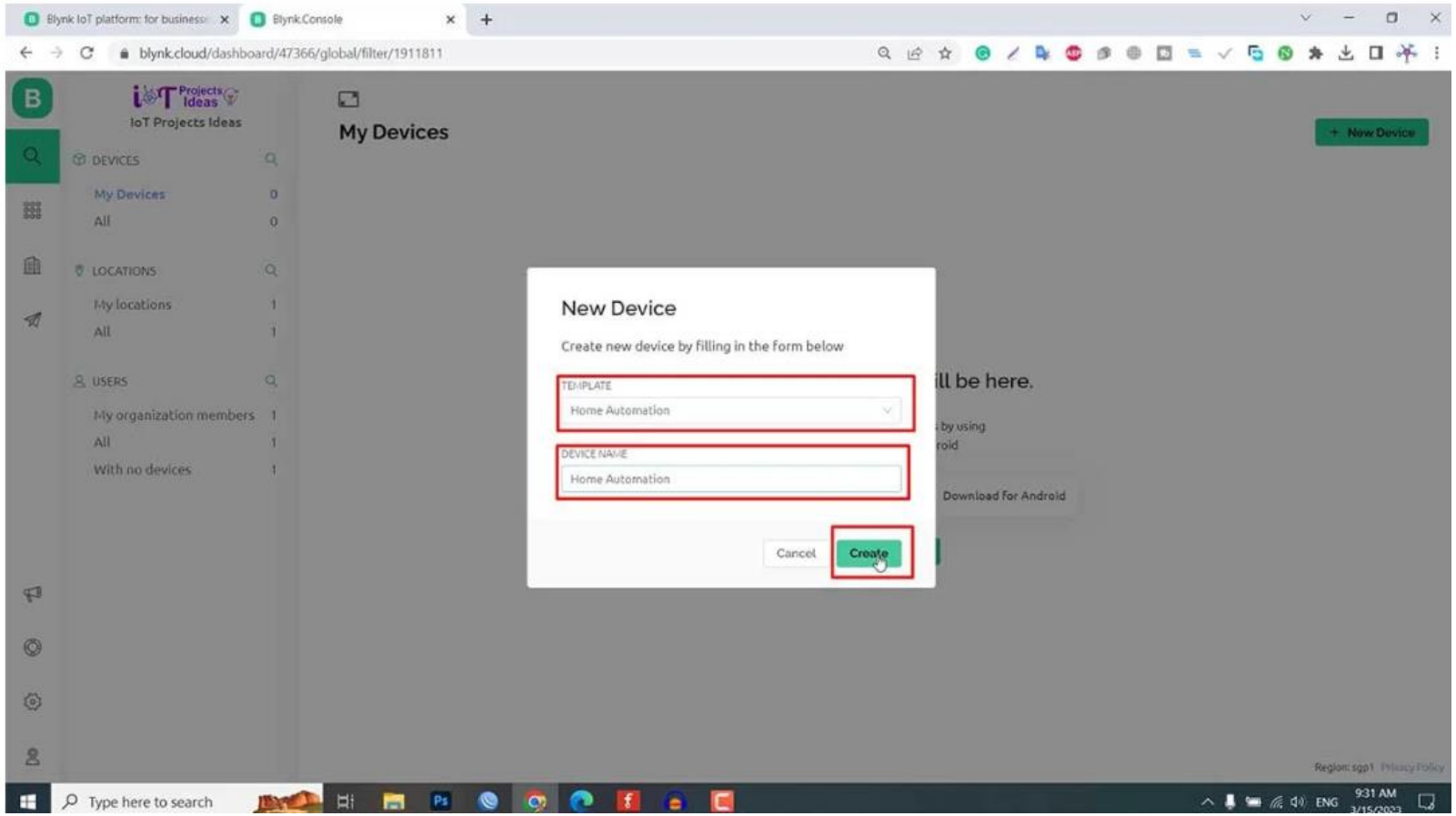
8. Adding New Device from Template



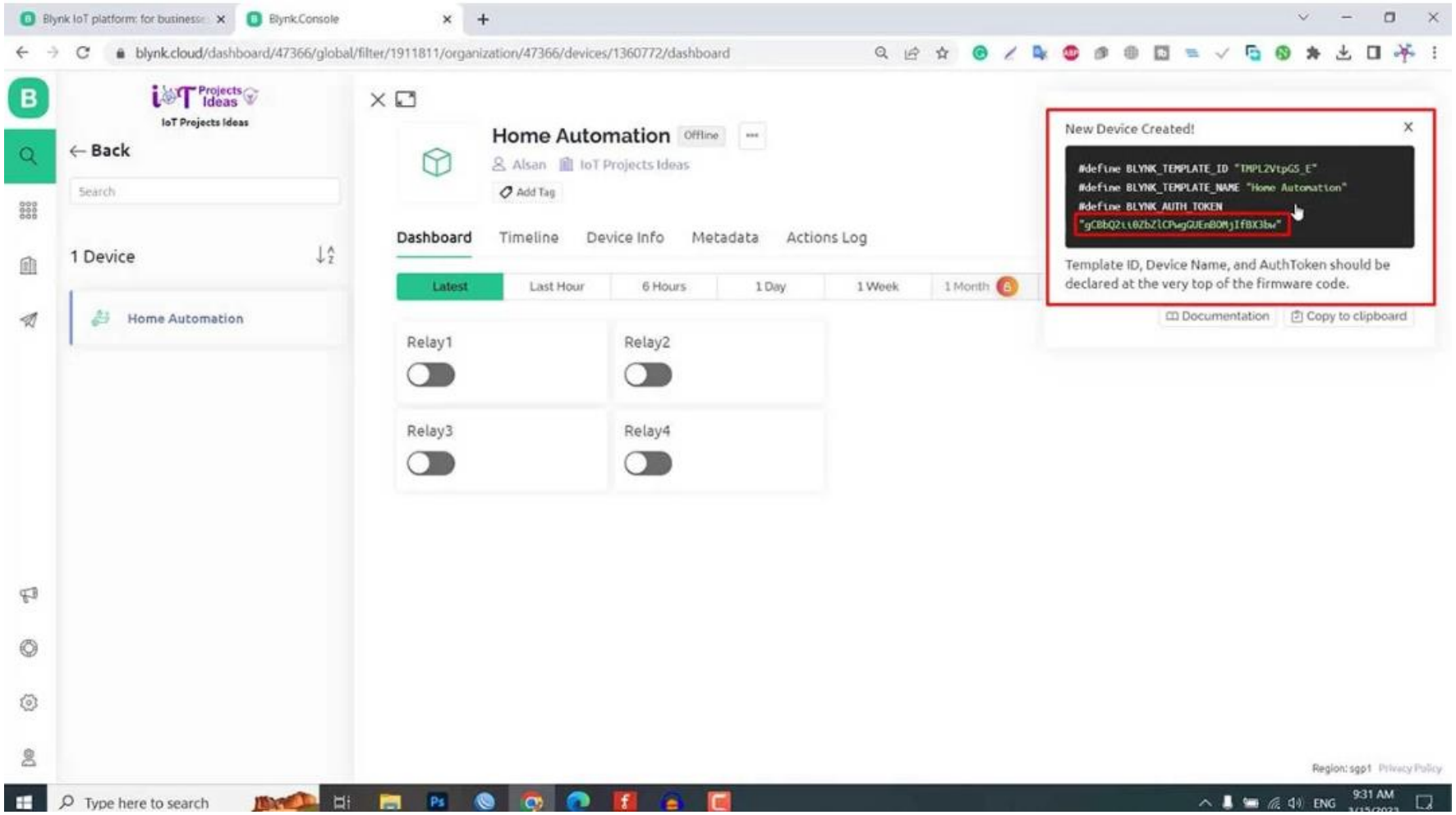
9. Next, we need to create a device for this project by selecting "**Choose from Template**" and then selecting the previously created template.



10. Give a name to the device and the new device will be created, displaying the **Blynk authentication token**.



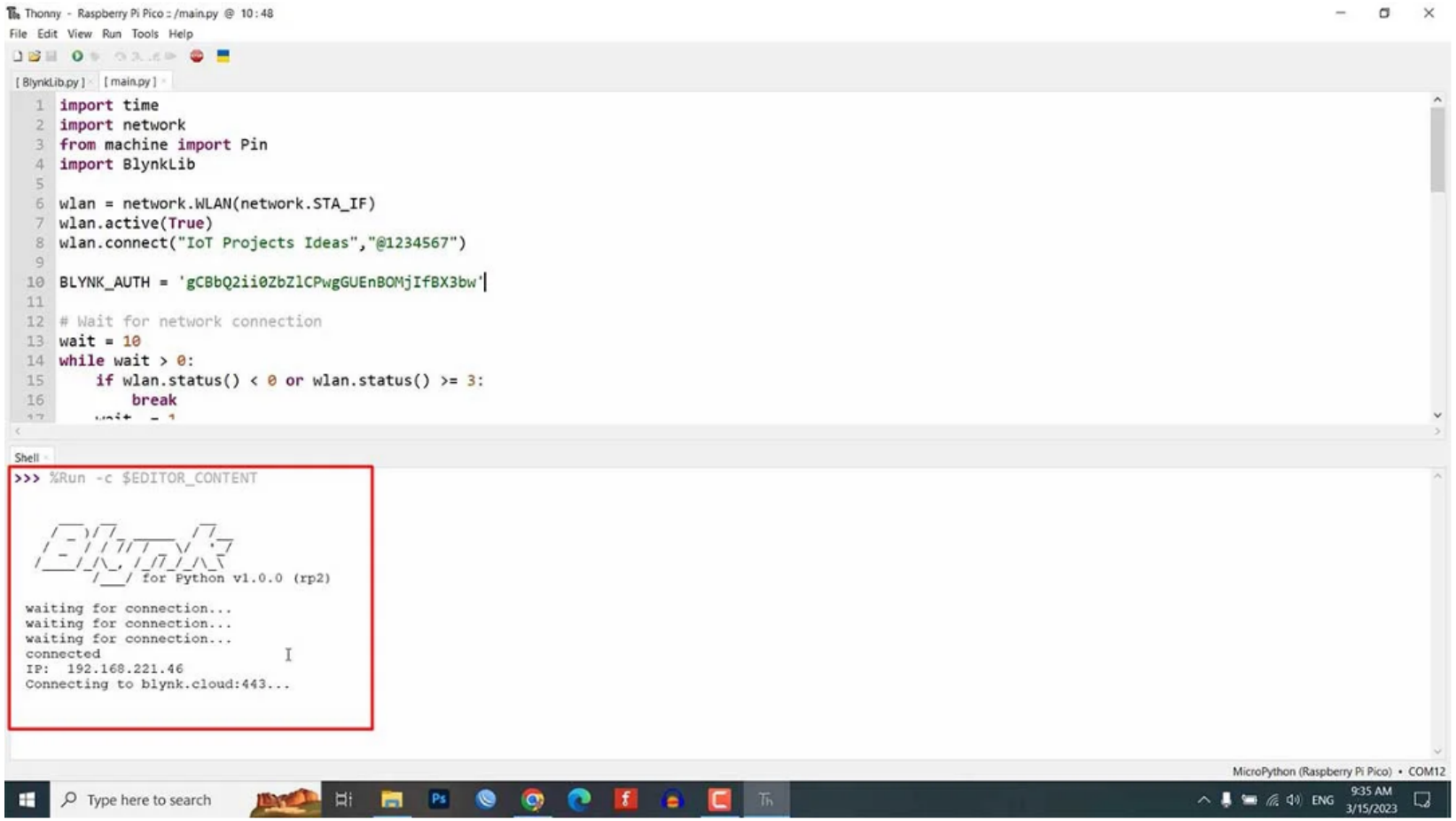
11. Copy this token as it is required for the **MicroPython code**. With the web dashboard now complete, you can control up to four home appliances for your **Blynk-based home automation project using the Raspberry Pi Pico W.**



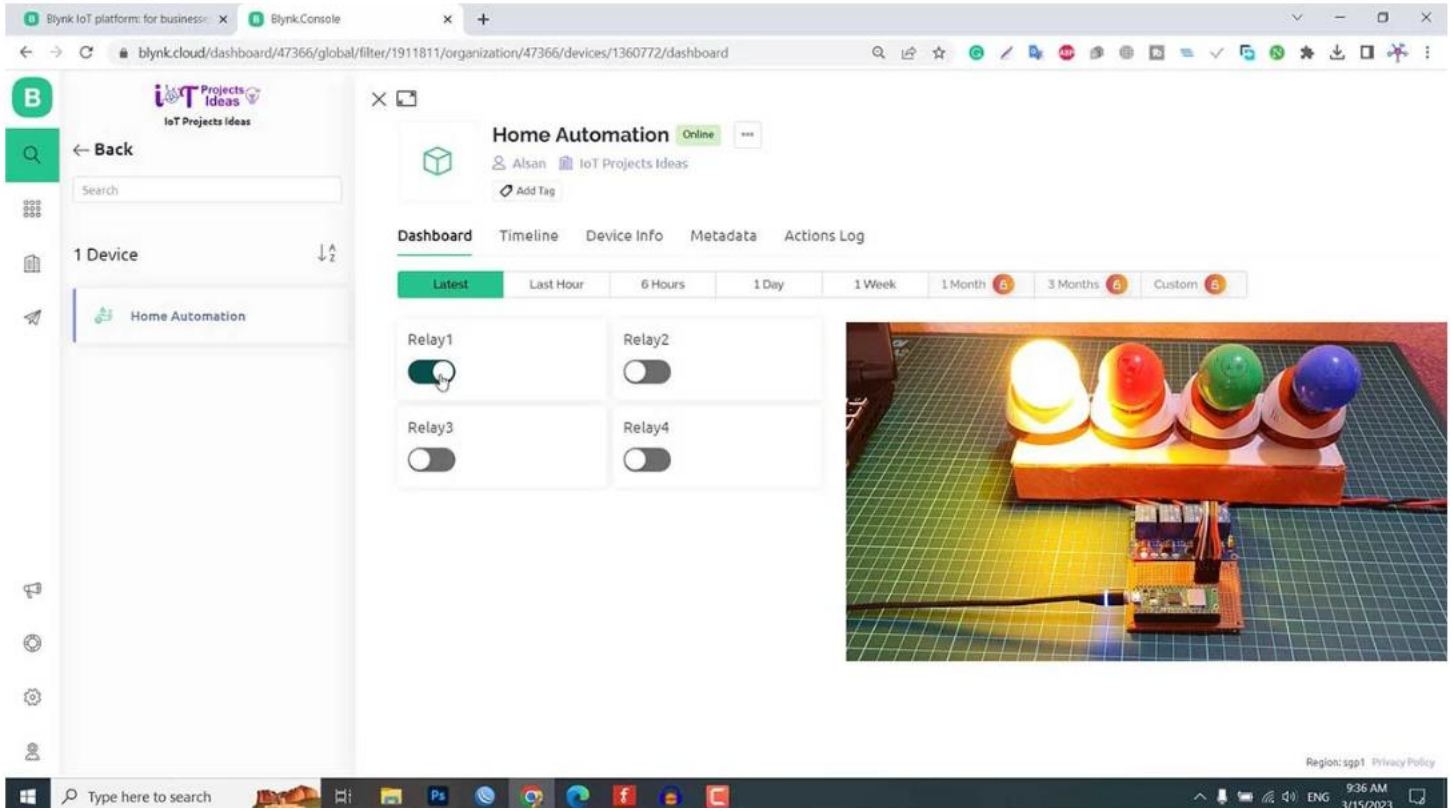
Source Code/Program

The program code for the Home Automation project using the **Raspberry Pi Pico W Blynk 2.0 app** consists of two parts: **BlynkLib.py and main.py**. To control the **Relays, the Blynk Library for MicroPython** is required.

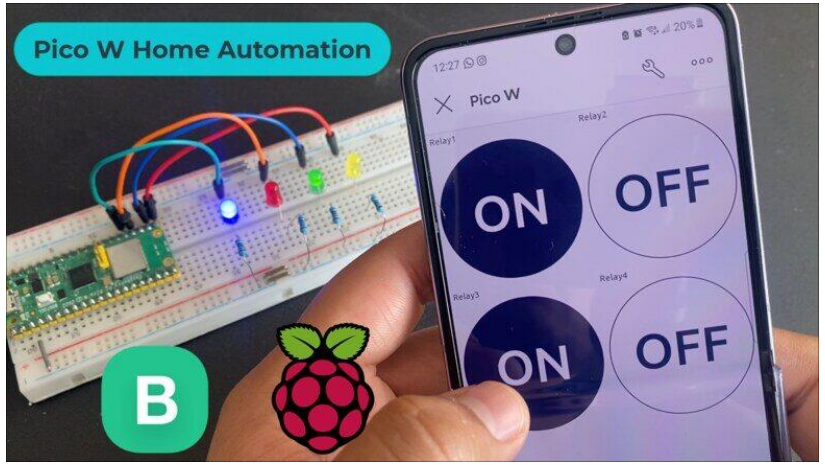
Output of the Code

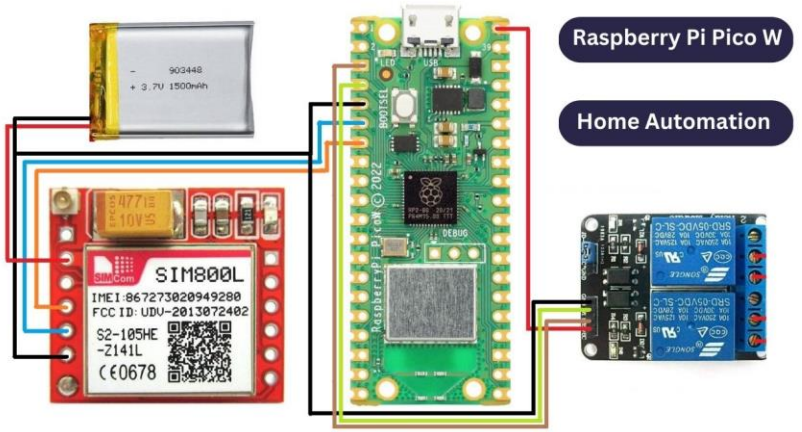


In the web app, navigate to the project dashboard and click on the **button widget**. This should turn on the **corresponding Relay**. Click the button again to turn off the Relay.

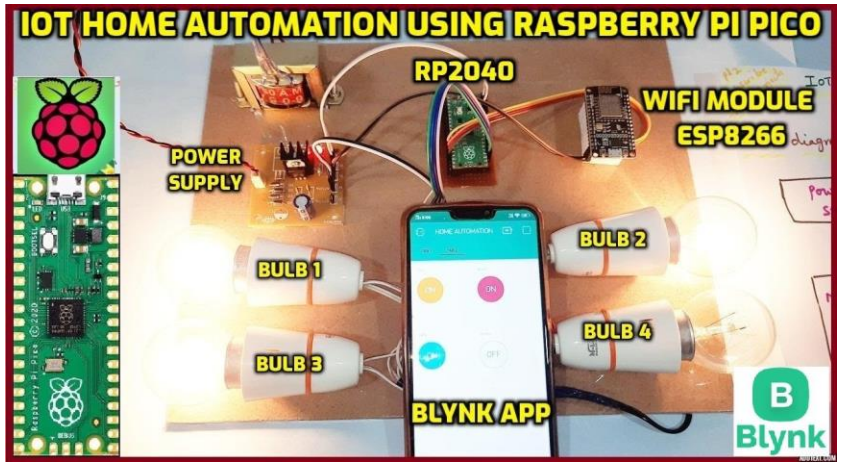


Home Automation with Raspberry Pi Pico W









Implantation IoT systems for Agriculture using cloud platforms Thingspeak

- IoT systems for smart agriculture can be successfully implemented with the Raspberry Pi Pico W and cloud platforms like Ubidots, Blynk, and ThingSpeak, delivering remote sensor monitoring, real-time control, and data-driven automation for irrigation, climate management, and crop health.
- Common agricultural sensors: Soil moisture, temperature (e.g., LM35, DHT11), humidity, and light sensors.
- Actuators: Water pumps, solenoid valves, relays for irrigation, fans, and auxiliary devices.
- Data acquisition performed by Pico W, which sends measurements to cloud dashboards and receives trigger commands.

Ubidots Implementation

- Pico W periodically reads sensor values (soil moisture, temperature, humidity) and sends them via HTTP REST API or MQTT to Ubidots cloud variables.
- The Ubidots dashboard visualizes soil state, environmental conditions, and appliance (pump, valve) status for farmers and agronomists.
- Ubidots events and logic blocks can send commands back to Pico W, automating irrigation when soil moisture is low or a threshold is crossed.

Blynk Implementation

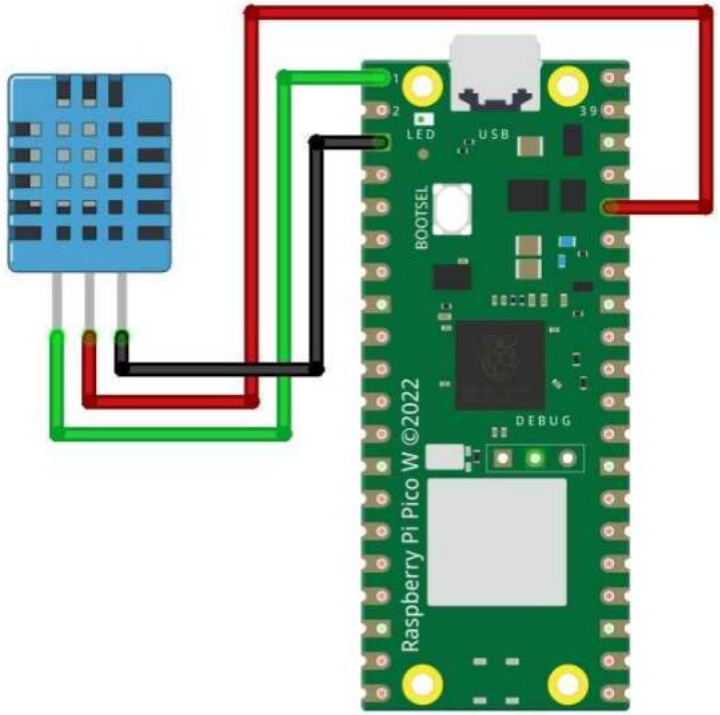
- Configure a Blynk dashboard on a mobile app to display soil data and to enable manual or automated control of pumps and relays.
- The Pico W runs Blynk client code and reacts to dashboard button presses (start/stop irrigation, activate fan) using Wi-Fi connectivity.
- Real-time feedback, historical charts, and alert notifications improve farm management efficiency and resource-saving.

ThingSpeak Implementation

- Sensor data streams from Pico W are sent to ThingSpeak channels for time-series analysis, plotting, and automated decision-making.
- Automated rules or MATLAB scripts in ThingSpeak can trigger irrigation commands or send SMS alerts when environmental factors fall outside optimal ranges.
- Easily scalable for multiple farm sectors addressed by separate Pico W units, centralizing information in comprehensive dashboards.

Example of Agriculture

Interfacing DHT11 Sensor with Raspberry Pi Pico W

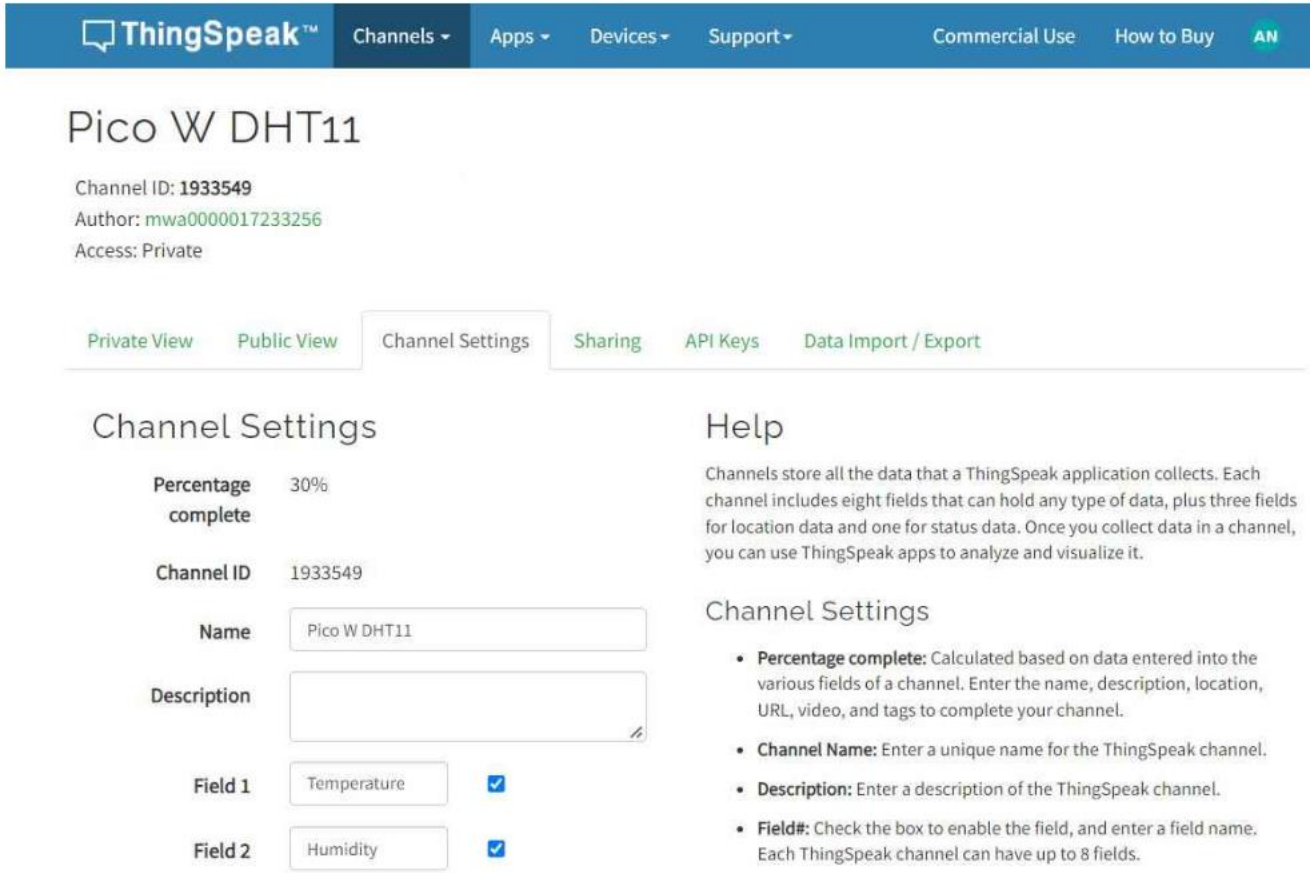


Steps for ThingSpeaks

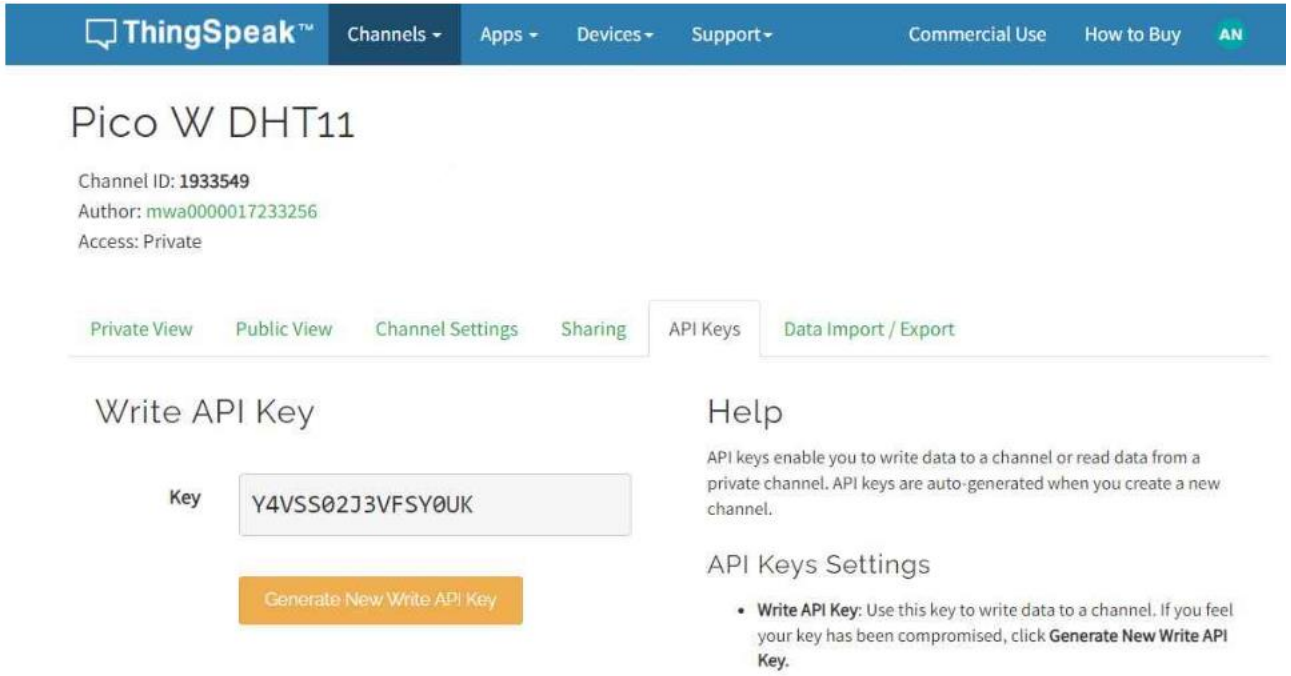
1. Visit https://thingspeak.com and create an account.



2. Then create a **new channel** and create two variables as shown in the image below.



3. Then create the **API keys**. This key is required for programming modifications and receiving data from Raspberry Pi Pico W.



4. Now click on channels so that you can see the online data streaming.

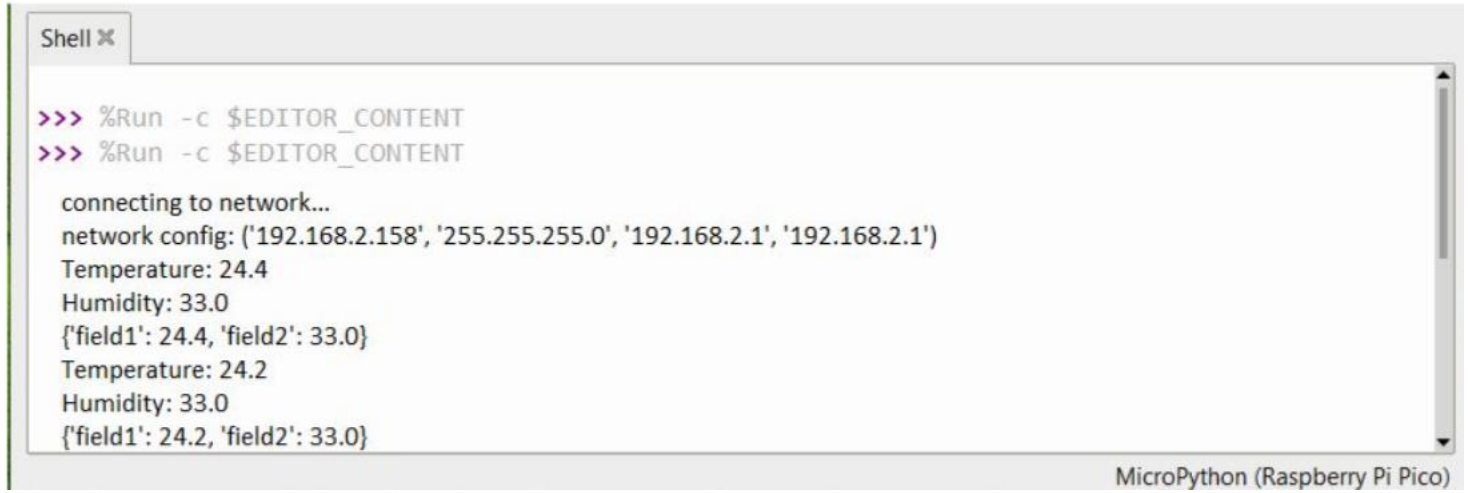
MicroPython Code

- Now let us write a **MicroPython code** for **Raspberry Pi Pico W** to send the DHT11 Humidity Temperature Sensor Data to the **Thingspeak** Server.
- To upload the data to Thingspeak Server, the Raspberry Pi Pico W needs to connect to the **Wi-Fi network**. Then using the **API Key** the Raspberry Pi Pico W uploads the humidity and temperature data to Thingspeak. You can observe the graphical data in **visualized** form in Thingspeak dashboard.
- The **MicroPython Code** is has two parts as the DHT11 Sensor requires a **library** for retrieving the humidity temperature data.

5. Monitor DHT11 Sensor Data on Thingspeak with Raspberry Pi Pico W

Run the library and also the main code.

The Raspberry Pi Pico W will connect to the **WiFi network** and establish connection with the Thingspeak Server. The **Python Shell** will display the event.



To have a visual representation of Temperature Humidity Data, you can go to the private view of Thingspeak Server **Dashboard**. The dashboard will have an entry of humidity temperature latest data after an interval of every 15 seconds.

ThingSpeak™ Channels Apps Devices Support Commercial Use How to Buy AN

Add Visualizations | Add Widgets | Export recent data | MATLAB Analysis | MATLAB Visualization

Channel Stats

Created: about 2 hours ago
Last entry: about a minute ago
Entries: 9

Field 1 Chart



Field 2 Chart



Implantation IoT systems for Industrial using cloud platforms Thingspeak

Goal: Monitor equipment health, environmental conditions, or production parameters to improve efficiency and predict failures.

Typical Sensors:

•Vibration Sensor

•Temperature Sensor (e.g., DS18B20)

•Gas/Smoke Sensors

•Pressure Sensors

Actuators:

•Cooling fans

•Alarm systems

•Machine control relays

Workflow:

•Sensors monitor machine/environment → Data sent via Raspberry Pi Pico W to cloud → Dashboard for real-time monitoring → Alerts or automation if thresholds exceeded.

Implementing a gas detection and oil leakage IoT system using Raspberry Pi Pico W and Thingspeak cloud platform

1.Hardware Setup:

A. Connect gas sensor (e.g., MQ-2) and oil leakage sensor to Raspberry Pi Pico W ADC pins.

B. Provide power (3.3V) and ground connections to sensors from the Pico W.

C. Connect the sensor outputs to the appropriate ADC pins on the Pico W (for example, GPIO 26 and 27).

2. Prepare the Software Environment:

1. Flash MicroPython firmware onto the Raspberry Pi Pico W.
2. Install and open the Thonny IDE or any MicroPython compatible editor.
3. Ensure the urequests library is available for making HTTP requests.

3. Configure Thingspeak Channel:

1. Create a free account on Thingspeak.
2. Create a new channel, name it (e.g., "Gas and Oil Leakage Monitoring").
3. Enable two fields, e.g., Field 1 for gas sensor, Field 2 for oil leakage sensor.
4. Note down the Channel ID and Write API Key.

4. Write MicroPython Code:

1. Connect Raspberry Pi Pico W to Wi-Fi using network credentials.
2. Read sensor data using the ADC pins.
3. Format and send sensor data to Thingspeak server using HTTP POST or GET requests with the API key.
4. Implement delay for periodic data transmission (Thingspeak allows updates every 15 seconds).

5. Upload and Run the Code on Raspberry Pi Pico W:

    1. Save the MicroPython code on Pico W.
    2. Run the code; Pico W connects to Wi-Fi and starts sending sensor data to Thingspeak.

6. Monitor Data on Thingspeak Dashboard:

    1. Log in to Thingspeak.
    2. Open your channel dashboard.
    3. View real-time gas and oil leakage sensor data in graphical formats.
    4. Set alert thresholds for critical sensor levels using Thingspeak integrations.

Code

```
import network
import urequests
import machine
import time

# Wi-Fi credentials
ssid = "YOUR_WIFI_SSID"
password = "YOUR_WIFI_PASSWORD"

# Thingspeak credentials
THINGSPEAK_WRITE_API_KEY =
'YOUR_THINGSPEAK_WRITE_API_KEY'
CHANNEL_ID =
"YOUR_THINGSPEAK_CHANNEL_ID"

# Initialize sensors on ADC pins
gas_sensor = machine.ADC(26)
oil_leak_sensor = machine.ADC(27)
```

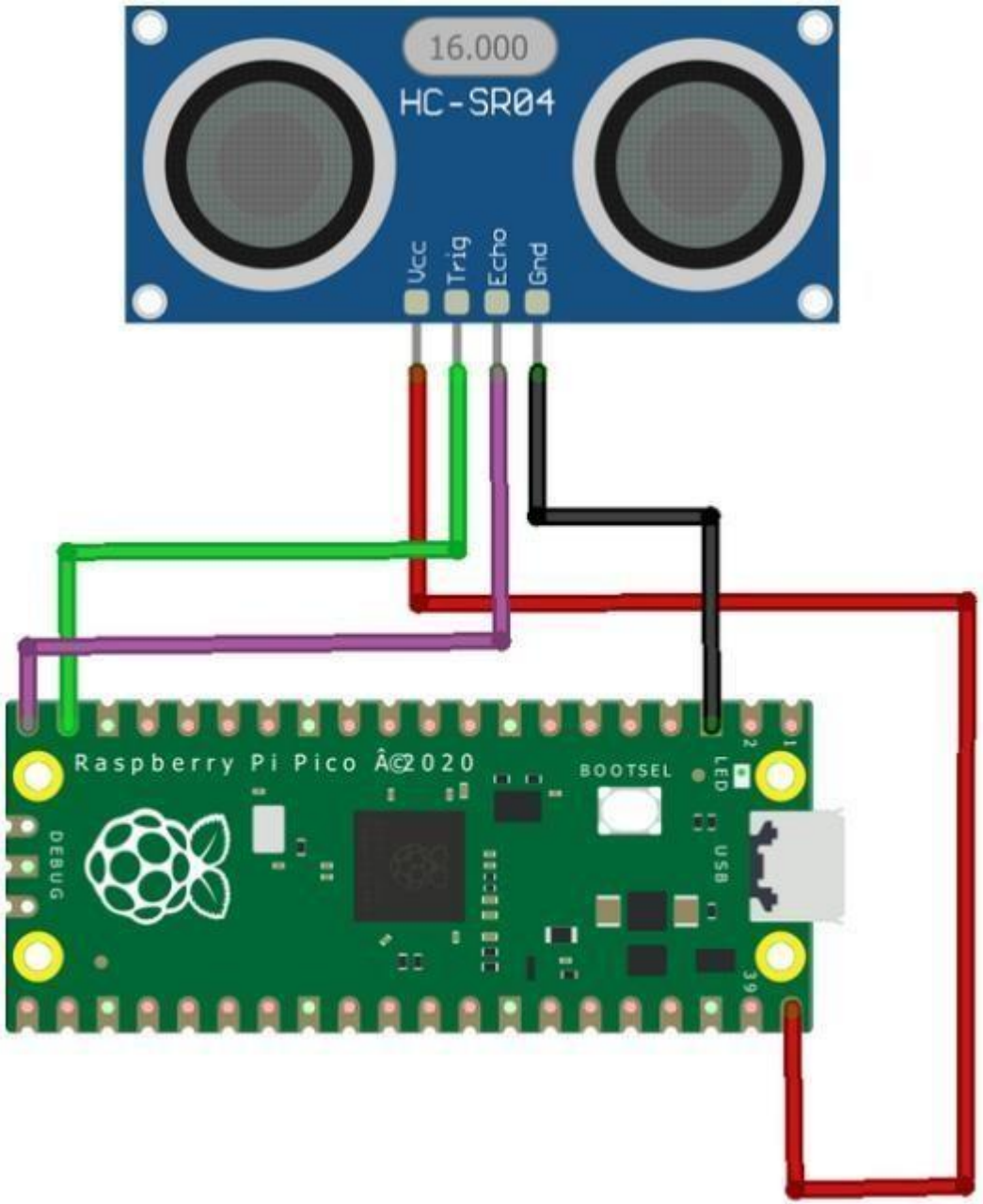
```
# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
if not wlan.isconnected():
    wlan.connect(ssid, password)
    while not wlan.isconnected():
        pass
print('Connected to WiFi. IP:', wlan.ifconfig()[0])
```

```
# Function to send data to Thingspeak
def send_data(gas_value, oil_value):
    url =
f"https://api.thingspeak.com/update?api_key={THINGSPEAK_WRITE_API_KEY}&field1={gas_value}&
field2={oil_value}"
    response = urequests.get(url)
    print('Data sent to Thingspeak:', response.text)
    response.close()

while True:
    gas_val = gas_sensor.read_u16()
    oil_val = oil_leak_sensor.read_u16()
    send_data(gas_val, oil_val)
    time.sleep(15)  # Thingspeak allows updates every 15 seconds
```

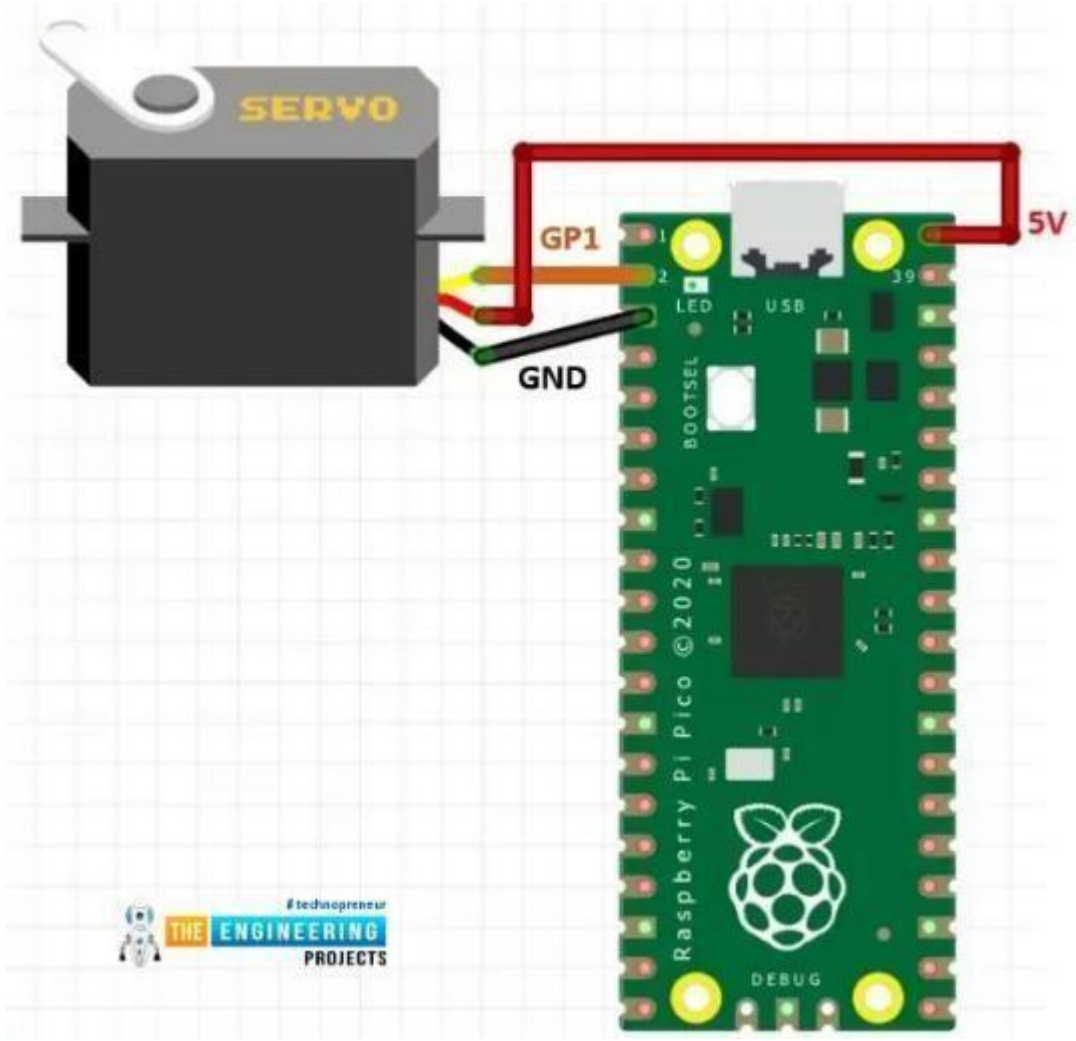
Platform	Strengths	How It Works
Ubidots	Industrial focus, easy dashboards, alerts	Upload sensor data via MQTT or HTTP API, build dashboards & alerts
Blynk	Mobile app-centric, easy remote control	Use virtual pins to read/write data, control actuators remotely via app
ThingSpeak	Data logging, MATLAB analytics integration	Send sensor data via HTTP, visualize in graphs, analyze trends

Ultrasonic Sensor Module With Raspberry Pi




```
from machine import Pin
import utime
trigger = Pin(14, Pin.OUT)
echo = Pin(15, Pin.IN)
def ultra():
  trigger.low()
  utime.sleep_us(2)
  trigger.high()
  utime.sleep_us(5)
  trigger.low()
  while echo.value() == 0:
     signaloff = utime.ticks_us()
  while echo.value() == 1:
     signalon = utime.ticks_us()
  timepassed = signalon - signaloff
  distance = (timepassed * 0.0343) / 2
  print("The distance from object is ",distance,"cm")
while True:
  ultra()
  utime.sleep(1)
```

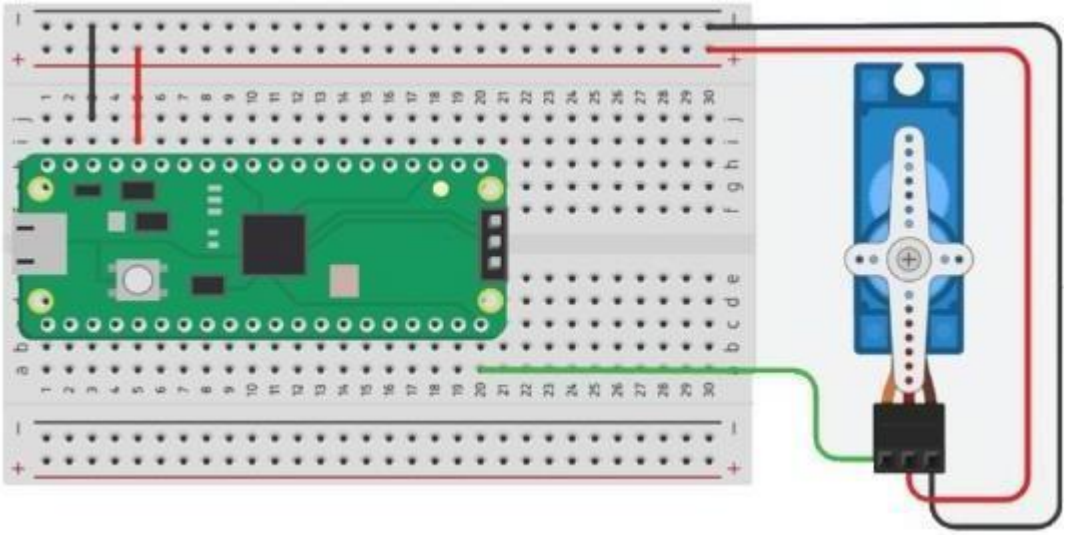
Servo Motor with Raspberry Pi Pico using MicroPython




```
from machine import Pin, PWM
import time
# declaring pwm object for servo motor pin i.e., GPIO_1
pwm = PWM(Pin(0))
# setting PWM frequency at 50Hz
pwm.freq(50)
while True:
        for position in range(1000, 9000, 50): # changing angular position
                pwm.duty_u16(position)
                time.sleep(0.1) # delay
        for position in range(9000, 1000, -50):
                pwm.duty_u16(position)
                time.sleep(0.1) # delay
```

Servo Motor With The Raspberry Pi Pico

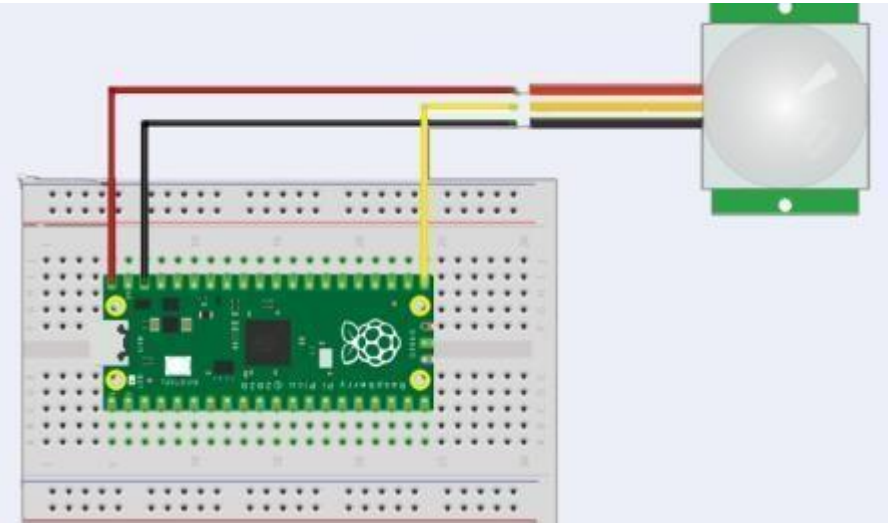




```
from machine import Pin, PWM

import utime;
Mid = 1500000
Min = 100000
Max = 200000
pwm = PWM(Pin(0))
pwm.freq(50)
Pwm.duty_ns(MID)
while True:
    pwm.duty_ns(Min)
    utime.sleep(1)
    pwm.duty_ns(Min)
    utime.sleep(1)
    pwm.duty_ns(Max)
    utime.sleep(1)
    pwm.duty_ns(Mid)
    utime.sleep(1)
```

PIR Sensor With The Raspberry Pi Pico



```
from machine import Pin
import utime

pir_sensor = Pin(16, Pin.IN)
utime.sleep(2)

while True:
  if pir_sensor.value() == 1:
      print("Motion Detected")
      utime.sleep(3)
  else:
      print("No Motion")
      utime.sleep(1)
utime.sleep(0.1)
```

Raspberry pi pico with 16x2 lcd interfacing

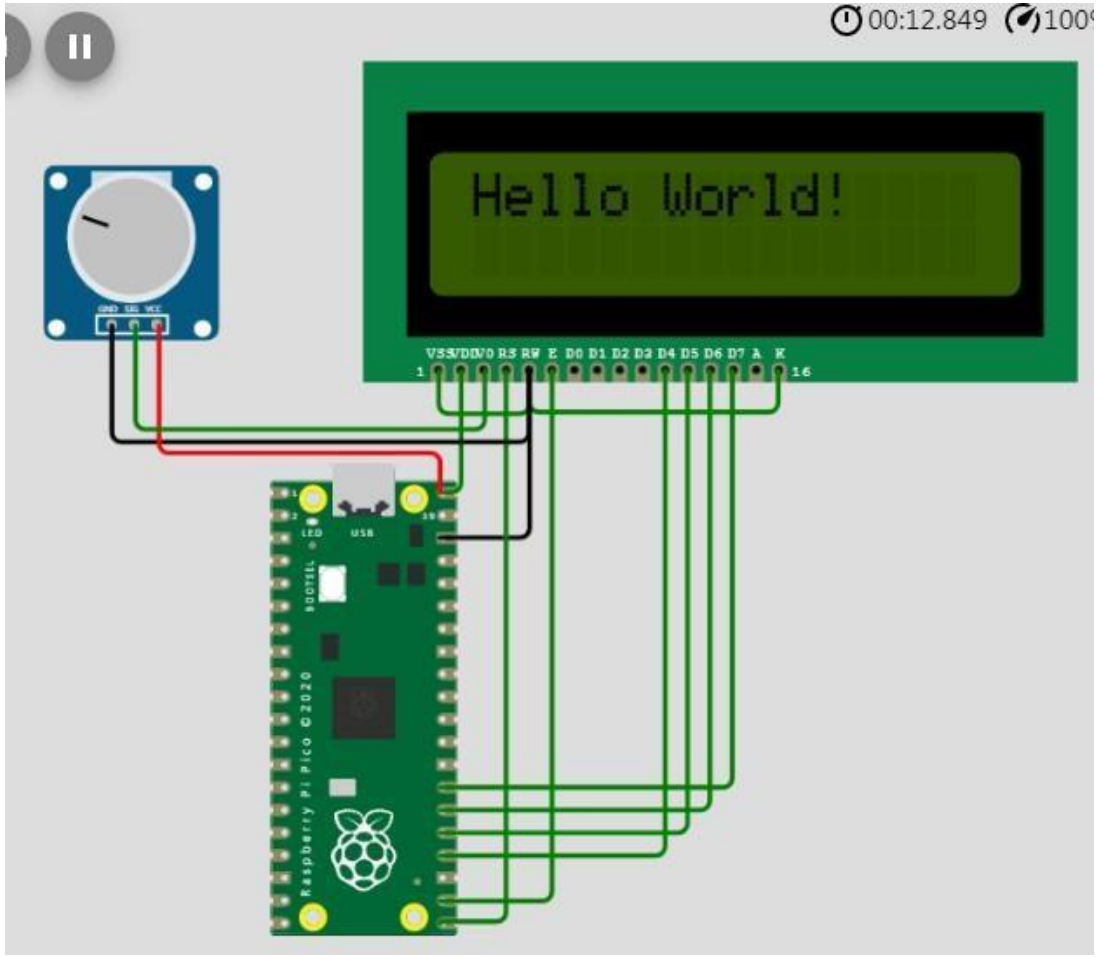
```
#print("Hello, Pi Pico!")
import machine
import utime

rs  =  machine.Pin(16,machine.Pin.OUT)
e   =   machine.Pin(17,machine.Pin.OUT)
d4  =  machine.Pin(18,machine.Pin.OUT)
d5  =  machine.Pin(19,machine.Pin.OUT)
d6  =  machine.Pin(20,machine.Pin.OUT)
d7 = machine.Pin(21,machine.Pin.OUT)

def pulseE():
    e.value(1)
    utime.sleep_us(40)
    e.value(0)
    utime.sleep_us(40)
def send2LCD4(BinNum):
    d4.value((BinNum & 0b00000001) >>0)
    d5.value((BinNum & 0b00000010) >>1)
    d6.value((BinNum & 0b00000100) >>2)
    d7.value((BinNum & 0b00001000) >>3)
    pulseE()
def send2LCD8(BinNum):
    d4.value((BinNum & 0b00010000) >>4)
    d5.value((BinNum & 0b00100000) >>5)
    d6.value((BinNum & 0b01000000) >>6)
    d7.value((BinNum & 0b10000000) >>7)
    pulseE()
    d4.value((BinNum & 0b00000001) >>0)
    d5.value((BinNum & 0b00000010) >>1)
    d6.value((BinNum & 0b00000100) >>2)
    d7.value((BinNum & 0b00001000) >>3)
    pulseE()
```

```
def setUpLCD():
    rs.value(0)
    send2LCD4(0b0011)#8 bit
    send2LCD4(0b0011)#8 bit
    send2LCD4(0b0011)#8 bit
    send2LCD4(0b0010)#4 bit
    send2LCD8(0b00101000)#4 bit,2 lines?,5*8 bots
    send2LCD8(0b00001100)#lcd on, blink off, cursor off.
    send2LCD8(0b00000110)#increment cursor, no display shift
    send2LCD8(0b00000001)#clear  screen
    utime.sleep_ms(2)#clear screen needs a long delay

setUpLCD()
rs.value(1)
for x in 'Hello World!':
    send2LCD8(ord(x))
```



Thank You